Leilão Distribuído com WebSocket

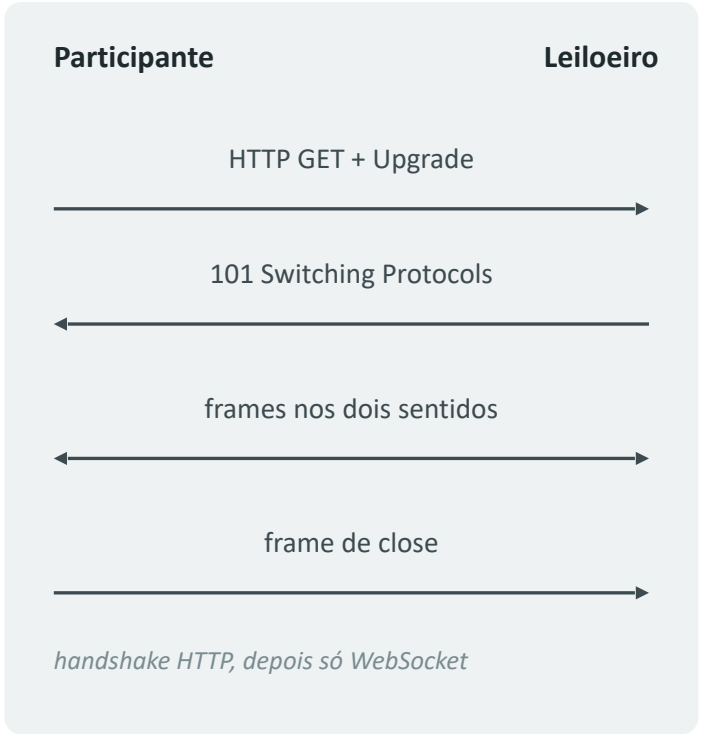
Sistemas Distribuídos e Computação Paralela · PUC-SP · Prof. Carlos Paes

Grupo 1: João Avila, Kauã Cordeiro, Luan Capella, Pedro Evaristo

Seminário 13/11/2026

WebSocket: um canal aberto nos dois sentidos

- Protocolo padronizado na RFC 6455 (2011), sobre uma única conexão TCP
- Começa como HTTP e faz upgrade: GET com "Upgrade: websocket" → resposta 101 Switching Protocols
- Full-duplex e persistente: cliente e servidor enviam quando quiserem, sem novo pedido
- Paradigma: troca de mensagens, comunicação direta entre processos
- Modelo assíncrono e orientado a eventos (open, message, close)



Vantagens, limitações e uso real

Vantagens

- Push em tempo real, sem polling
- Baixa sobrecarga por mensagem depois do handshake
- Funciona no navegador e passa por portas HTTP (80/443)

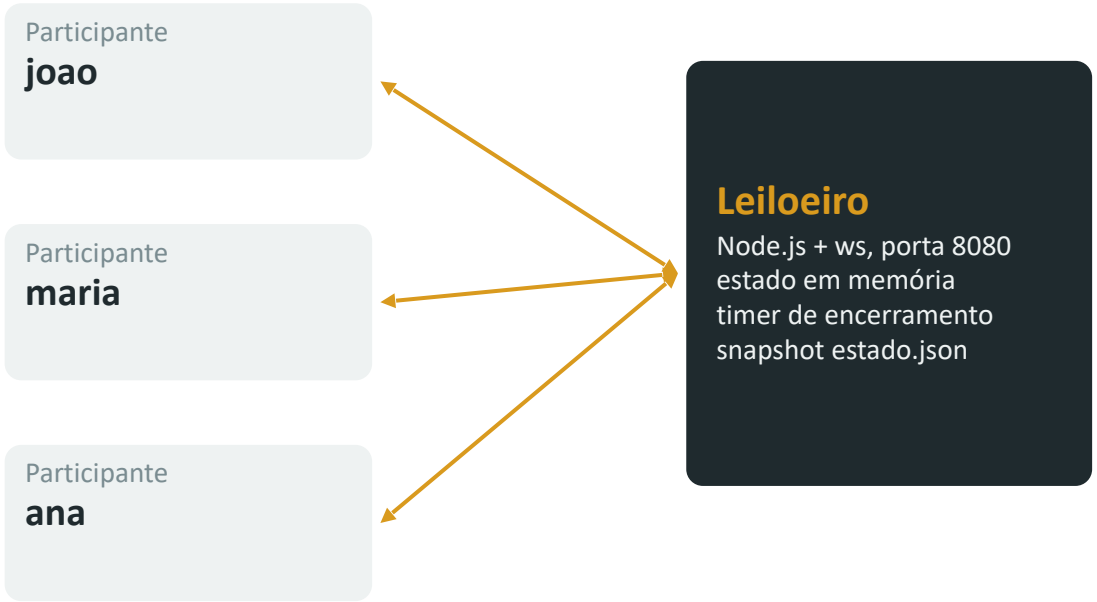
Limitações

- Sem reconexão nem garantia de entrega: fica com a aplicação
- Conexão com estado: escalar exige cuidado (sticky sessions, pub/sub)
- Sem contrato formal de mensagens (formato é por nossa conta)

Uso real

- Discord: Gateway de eventos via WebSocket
- Slack: Socket Mode para apps
- Corretoras (ex.: Coinbase): feed de cotações
- Ferramentas: ws, Socket.IO, websockets (Python)

Arquitetura da POC



Mensagens (JSON, campo "type")

Participante → Leiloeiro

`JOIN · GET_STATE · BID`

Leiloeiro → Participante

`STATE · BID_RESULT`
`NEW_HIGHEST_BID (push)`
`AUCTION_CLOSED · ERROR`

```
{"type":"BID","bidder":"joao",
 "amount":150}
```

Participante abre a conexão; depois os dois lados enviam livremente

Vídeo: demonstração e experimentos

1	**Funcionamento normal**	lances recusados e aceitos, push para todos, encerramento
2	**Queda do Leiloeiro (RE2)**	kill -9, código 1006 e estado restaurado ao reiniciar
3	**Lances simultâneos (RE3)**	mesmo valor ao mesmo tempo: o primeiro a chegar ganha

Como usamos IA

Onde ajudou

- Roteiro da POC e formato das mensagens a partir da especificação
- Implementação do Leiloeiro, Participante e script do RE3 (Claude Code)
- Estrutura do README, do IA.md e destes slides

Onde errou e como corrigimos

- Script do RE3 travava: a resposta STATE chegava antes de o código começar a esperar por ela. Pegamos rodando o teste; corrigido esperando o STATE dentro do connect
- Participante quebrava ao fechar a entrada (readline usado depois de fechado). Corrigido com uma flag
- Todo código gerado foi testado rodando antes do commit

Conclusão

Usar WebSocket quando…

- os clientes precisam saber na hora (lances, chat, cotações)
- há muitas mensagens pequenas nos dois sentidos
- o cliente é um navegador

Evitar quando…

- consultas eventuais bastam (REST é mais simples)
- é preciso entrega garantida ou desacoplamento (MOM)
- queremos contrato formal de interface (gRPC, SOAP)

No nosso leilão apareceram: ordem dos lances decidida pela chegada ao servidor, o Leiloeiro como ponto único de falha e o acoplamento de uma conexão aberta por Participante.